CHỦ ĐỀ HỆ THỐNG QUẢN LÝ THIẾT BỊ VÀ HỢP ĐỒNG BẢO TRÌ

1. Mô tả nghiệp vụ

Một trường đại học cần xây dựng chương trình quản lý các thiết bị phục vụ học tập và giảng dạy. Hệ thống bước đầu quản lý ba loại thiết bị:

- <Điều khiển điều hòa>.
- <Micro>.
- <Điều khiển máy chiếu>.

Mỗi thiết bị có mã, tên, trạng thái và <số tiết> được phép mượn tối đa. Một phiếu mượn thuộc về một người mượn và có thể chứa nhiều thiết bị.

Trong quá trình xử lý có thể xảy ra các tình huống bất thường:

- Thiết bị không ở trạng thái sẵn sàng.
- Một thiết bị được thêm hai lần vào cùng phiếu.
- Người mượn vượt quá số thiết bị cho phép.
- Thời hạn mượn lớn hơn giới hạn của thiết bị.
- Trả thiết bị không thuộc phiếu mượn.
- Trả một thiết bị đã được trả trước đó.
- Ngày trả không hợp lệ.

Hệ thống phải sử dụng ngoại lệ để thông báo lỗi. Khi một thao tác thất bại, trạng thái thiết bị và phiếu mượn phải được giữ nhất quán

2. Yêu cầu thiết kế

Trạng thái thiết bị

2.1 Sử dụng liệt kê – enum, vd

```
public enum DeviceStatus
{
   Available,
   Borrowed,
   UnderMaintenance
}
```

2.2. Lớp trừu tượng Device

Xây dựng lớp trừu tượng Device gồm ít nhất các thông tin:

- Mã thiết bị.
- Tên thiết bị.
- Năm đưa vào sử dụng.
- Giá mua.

- Trạng thái hoạt động.

Yêu cầu:

- Mã thiết bị không được rỗng.
- Giá mua phải lớn hơn 0.
- Năm đưa vào sử dụng không được lớn hơn năm hiện tại.
- Mã thiết bị chỉ được thiết lập khi khởi tạo.
- Cung cấp phương thức trừu tượng tính **chi phí bảo trì dự kiến trong một năm**.
- Ghi đè phương thức biểu diễn thông tin của thiết bị dưới dạng chuỗi.

Có thể sử dụng enum cho trạng thái:

```
public enum DeviceStatus
{
   Active,
   UnderMaintenance,
   Retired
}
```

Các lớp dẫn xuất

3. Yêu cầu sử dụng kết tập

Xây dựng lớp ClassRoom gồm:

- Mã phòng.
- Tên phòng.
- Sức chứa.
- Danh sách thiết bị.

Các phương thức tối thiểu:

```
void AddDevice(Device device);
bool RemoveDevice(string deviceId);
Device? FindDevice(string deviceId);
decimal CalculateAnnualMaintenanceCost();
List<Device> GetDevicesRequiringMaintenance();
```

Quy tắc:

1. Không thêm hai thiết bị có cùng mã vào một phòng.
2. Không chấp nhận đối tượng thiết bị null.
3. CalculateAnnualMaintenanceCost() phải sử dụng tính đa hình, không kiểm tra lần lượt bằng if hoặc switch xem thiết bị thuộc lớp nào.
4. GetDevicesRequiringMaintenance() trả về các thiết bị:
    - Đang ở trạng thái UnderMaintenance; hoặc
    - Có thời gian sử dụng trên 5 năm.

5. Người mượn và interface

Xây dựng interface:

```
public interface IBorrower
{
   string BorrowerId { get; }
   string FullName { get; }

   int GetMaximumDevices();
}
```

Xây dựng hai lớp thực thi interface:

Lecturer

- Giảng viên được mượn tối đa **3 thiết bị** trên một phiếu.

TeachingAssistant

- Trợ giảng được mượn tối đa **2 thiết bị** trên một phiếu.

6. Lớp BorrowItem

```
public class BorrowItem
{
   public TeachingDevice Device { get; }
   public DateTime BorrowedDate { get; }
   public int fromPeriod { get; set}
   public int toPeriod { get; set}
   public DateTime? ReturnedDate { get; private set; }

   public bool IsReturned { get; }

   public void Return(DateTime returnedDate);
}
```

Quy tắc

- Device không được null.
- DueDate phải sau hoặc bằng BorrowedDate.
- Tương tự với tiết mượn từ tiết đến tiết
- ReturnedDate ban đầu là null.
- Không được trả một BorrowItem hai lần.
- Ngày trả không được trước ngày mượn.

7. Lớp BorrowingSlip

```
public class BorrowingSlip
{
   public string SlipId { get; }
   public IBorrower Borrower { get; }
   public string TeachingRoom { get; }
   public DateTime BorrowedDate { get; }

   public IReadOnlyList<BorrowItem> Items { get; }

   public void AddDevice(TeachingDevice device, int borrowDays);
   public decimal ReturnDevice(string deviceId, DateTime returnedDate);
   public decimal CalculateTotalLateFee();
}
```

Quy tắc chung

- SlipId, Borrower và TeachingRoom phải hợp lệ.
- BorrowingSlip hợp thành các BorrowItem.
- BorrowItem chỉ tham chiếu đến TeachingDevice; xóa phiếu không xóa thiết bị khỏi hệ thống.
- Người mượn vẫn tồn tại độc lập khi phiếu bị xóa.

8. Hệ phân cấp ngoại lệ nghiệp vụ

8.1. Ngoại lệ cơ sở

```
public class BorrowingException : Exception
{
   public BorrowingException(string message)
      : base(message)
   {
   }

   public BorrowingException(
      string message,
      Exception innerException)
      : base(message, innerException)
   {
   }
}
```

8.2. Các ngoại lệ dẫn xuất bắt buộc

DeviceUnavailableException

Phát sinh khi thiết bị không ở trạng thái Available.

Nên chứa:

```
public string DeviceId { get; }
public DeviceStatus CurrentStatus { get; }
```

BorrowLimitExceededException

Phát sinh khi người mượn đã đạt số thiết bị tối đa.

Nên chứa:

```
public string BorrowerId { get; }
public int MaximumDevices { get; }
```

InvalidBorrowPeriodException

Phát sinh khi:

- Số ngày mượn không dương.

Nên chứa mã thiết bị, số ngày yêu cầu và số ngày tối đa.

DuplicateDeviceException

Phát sinh khi cùng một thiết bị đã tồn tại trong phiếu.

DeviceNotInSlipException

Phát sinh khi yêu cầu trả một thiết bị không thuộc phiếu.

DeviceAlreadyReturnedException

Phát sinh khi thiết bị trong phiếu đã được trả trước đó.

Sinh viên có thể bổ sung ngoại lệ khác nếu giải thích được lý do.

9. Quy tắc xử lý nghiệp vụ

9.1. Thêm thiết bị vào phiếu

```
public void AddDevice(
   TeachingDevice device,
   int borrowDays)
```

Phương thức phải kiểm tra tối thiểu:

1. device không được null.
2. borrowDays phải lớn hơn 0.
3. Thiết bị chưa tồn tại trong phiếu.
4. Người mượn chưa đạt giới hạn thiết bị.
5. Thiết bị đang Available.

Chỉ sau khi các điều kiện hợp lệ, chương trình mới:

1. Tạo BorrowItem.
2. Chuyển thiết bị thành Borrowed.
3. Thêm item vào phiếu.

9.2. Bảo đảm tính nhất quán

Nếu AddDevice() thất bại:

- Thiết bị không bị chuyển sai thành Borrowed.
- Danh sách Items không bị thêm một phần.
- Phiếu giữ nguyên trạng thái trước thao tác.

Cách làm không phù hợp:

```
public void AddDevice(
   TeachingDevice device,
   int borrowDays)
{
   device.MarkAsBorrowed();

   if (items.Count >= Borrower.GetMaximumDevices())
   {
      throw new BorrowLimitExceededException(...);
   }

   items.Add(new BorrowItem(...));
}
```

Hướng an toàn hơn:

```
public void AddDevice(
    TeachingDevice device,
    int borrowDays)
{
    ValidateBorrowing(device, borrowDays);

    var item = new BorrowItem(
        device,
        BorrowedDate,
        BorrowedDate.AddDays(borrowDays));

    device.MarkAsBorrowed();
    items.Add(item);
}
```

9.3. Trả thiết bị

```
public decimal ReturnDevice(
    string deviceId,
    DateTime returnedDate)
```

Phương thức thực hiện:

1. Kiểm tra deviceId.
2. Tìm BorrowItem theo mã thiết bị.
3. Nếu không tìm thấy, phát sinh DeviceNotInSlipException.
4. Nếu item đã được trả, phát sinh DeviceAlreadyReturnedException.
5. Kiểm tra ngày trả.
6. Ghi nhận ngày trả.
7. Chuyển thiết bị về Available.

10. Sử dụng try, catch và finally

Lớp nghiệp vụ phát sinh ngoại lệ. Lớp ứng dụng hoặc Main() bắt ngoại lệ để thông báo và quyết định luồng xử lý.

11. Tình huống kiểm thử

11.1. Luồng thành công

1. Tạo một giảng viên.
2. Tạo một điều khiển máy chiếu, một điều khiển điều hòa và một micro không dây.
3. Tạo phiếu mượn cho một phòng dạy.
4. Thêm điều khiển máy chiếu và micro vào phiếu.
5. Trả điều khiển máy chiếu đúng hạn.
6. Trả micro muộn 2 ngày.
7. Hiển thị các phiếu trả muộn, chưa trả.

11.2. Luồng phát sinh ngoại lệ

Kiểm thử ít nhất năm tình huống:

1. Mượn điều khiển điều hòa đang UnderMaintenance.
2. Trợ giảng mượn thiết bị thứ ba.
3. Thêm cùng một micro hai lần.
4. Mượn micro trong 3 ngày dù giới hạn là 1 ngày.
5. Trả một điều khiển máy chiếu không thuộc phiếu.
6. Trả cùng một thiết bị hai lần.
7. Trả thiết bị với ngày trả trước ngày mượn.